

TEMA 1: SELECCIÓN DE ARQUITECTURAS Y HERRAMIENTAS DE PROGRAMACIÓN

Módulo Profesional: Desarrollo Web en Entorno Cliente (2º DAW) | **Ciclo Formativo:** Desarrollo de Aplicaciones Web

1. INTRODUCCIÓN Y DESARROLLO DE SOFTWARE

El mundo de la informática experimentó un crecimiento exponencial con la llegada de internet. El acceso a la información se democratizó y se extendió a todos los rincones del planeta. Tal fue la importancia de esta innovación tecnológica que se popularizó la expresión «si no estás en la web, no existes».

Pero los sitios web no nacen por generación espontánea; necesitan profesionales que los desarrollen e infraestructura que los soporte. Desde aquellos primeros días hasta hoy, el cambio y la especialización han sido tan radicales que hasta el concepto de desarrollo web parece demasiado amplio para abarcar todas las tecnologías que intervienen. Por todo ello, se hace necesario situar el contexto, conocer la foto global y poner el foco en aquellos conceptos imprescindibles para comenzar a caminar en la apasionante tarea del desarrollo web.

1.1. Concepto y Ciclo de Vida del Software

De forma resumida puede definirse el **desarrollo de software** como el proceso de creación de un programa informático o un conjunto de programas para realizar las diferentes tareas que cubren una necesidad empresarial u operativa (automatización, generación de informes, venta online, etc.). A pesar de la heterogeneidad de las necesidades, todas comparten dos objetivos comunes: **automatización** y **aumento de la eficiencia**.

Cada proceso de creación de aplicaciones sigue sistemáticamente una serie de fases estructuradas:

- **1. Planificación y análisis de requisitos:** Es una de las etapas más importantes. La realizan los miembros más experimentados del equipo junto con el cliente para estudiar la viabilidad económica, operativa y técnica, además de identificar riesgos.
- **2. Definición de requisitos:** Se definen y documentan claramente los requisitos acordados con el cliente, los cuales servirán de base para el diseño y la implementación.
- **3. Diseño de la arquitectura:** Se aborda la solución definiendo la estructura interna de los módulos, su comunicación, flujo de datos con sistemas externos y los estándares de calidad.
- **4. Construcción o implementación:** Fase de generación de código utilizando lenguajes, compiladores, intérpretes y depuradores según las especificaciones del diseño.
- **5. Pruebas:** Se detectan, informan, rastrean y reparan los defectos del producto hasta alcanzar los estándares de calidad exigidos.
- **6. Despliegue y mantenimiento:** Se despliega la aplicación en el entorno de producción y se aplican las correcciones y modificaciones necesarias para mantener su operatividad eficiente.

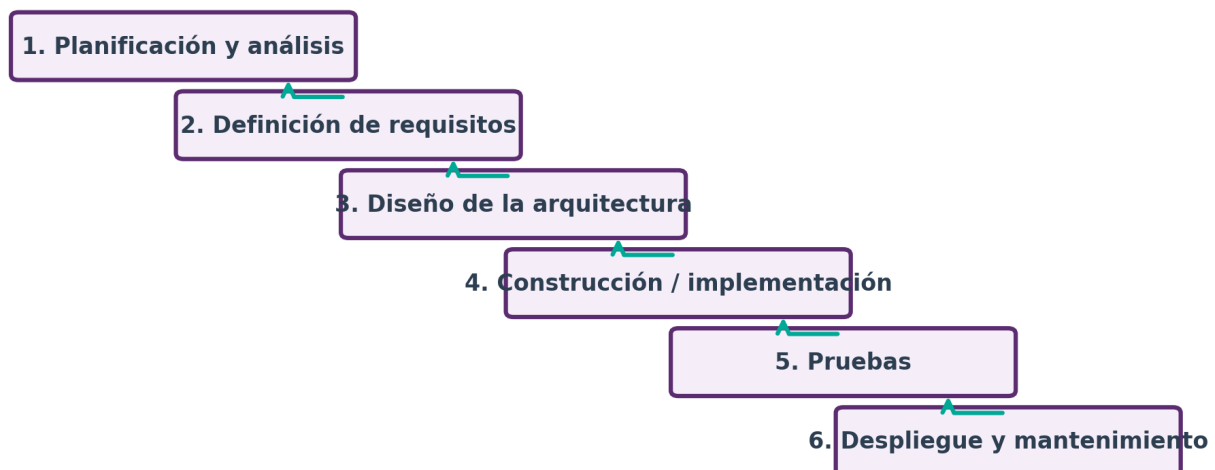


Figura 1.1. Ciclo de desarrollo de software tradicional en cascada.

1.1.1. Tipos de Aplicaciones

Las clasificaciones de software han evolucionado significativamente. En la actualidad, una misma aplicación puede dar servicio a múltiples tipos de dispositivos. Los enfoques de desarrollo principales son:

- **Desarrollo de aplicaciones personalizadas:** Cuando el software comercial no cumple los requisitos específicos de la organización.
- **Desarrollo rápido de aplicaciones (RAD):** Enfoque incremental donde se desarrollan módulos de forma simultánea para acortar tiempos.
- **Desarrollo de aplicaciones low-code:** Plataformas con interfaces visuales intuitivas que permiten crear aplicaciones mediante arrastrar y soltar con mínima intervención de código.
- **Desarrollo de aplicaciones móviles:** Orientado a la creación de software para plataformas móviles nativas o híbridas.
- **Desarrollo de aplicaciones de bases de datos:** Diseñadas para recopilar, organizar, calcular y administrar grandes volúmenes de información.
- **Desarrollo de aplicaciones empresariales:** Software a gran escala para gestionar procesos de negocio complejos e integración masiva.
- **Desarrollo de aplicaciones web:** Software alojado en servidores web al que se accede mediante navegador desde cualquier dispositivo conectado.

2. ARQUITECTURA WEB: LADO CLIENTE Y LADO SERVIDOR

El desarrollo web se divide fundamentalmente en dos grandes áreas: **front-end** (lado del usuario/cliente) y **back-end** (lado de la máquina/servidor):

- **Lado cliente:** Comprende todo aquello con lo que el usuario interactúa directamente en el navegador. El objetivo de los desarrolladores front-end es programar la interfaz de usuario (UI) y garantizar una experiencia de usuario (UX) óptima.
- **Lado servidor:** Procesa las acciones del usuario, ejecuta la lógica de negocio, interactúa con las bases de datos y devuelve las respuestas estructuradas al cliente.

Tabla 1.1. Principales diferencias entre el desarrollo de los lados cliente y servidor.

Criterio	Lado Cliente (Front-end)	Lado Servidor (Back-end)
Definición	Implementación efectiva de los componentes visuales de la aplicación web.	Implementación de la lógica de negocio, acceso a datos y administración del servidor.
Habilidades requeridas	HTML, CSS, SASS, JavaScript...	Python, Ruby, Java, PHP, Node.js...
Independencia	No puede funcionar de forma independiente excepto en sitios estáticos.	Funciona de forma independiente respecto al front-end.
Objetivo	Garantizar que todos los usuarios accedan fácilmente y la app responda en cualquier dispositivo.	Garantizar ejecución escalable, eficiente, sin fallos y con baja latencia.
Equipo de desarrollo	Diseñar y desarrollar la apariencia e interactividad según la entrada del usuario.	Proporcionar datos al front-end, vincular páginas, brindar seguridad y soporte.
Frameworks principales	AngularJS, React, Vue.js...	Django, Flask, Laravel, Ruby on Rails, Spring...
Habilidades adicionales	Comprensión profunda de diseño UI (User Interface) y UX (User Experience).	Razonamiento lógico estructurado y resolución de problemas complejos.

Nota Técnica: Framework, UI y UX

- **Framework:** Conjunto de herramientas y librerías software que proporcionan utilidades estandarizadas para desarrollar aplicaciones de forma escalable y ágil.
- **UI (User Interface):** Elementos visuales (botones, menús, controles) diseñados para la interacción.
- **UX (User Experience):** Grado de satisfacción del usuario al interactuar con la aplicación, buscando que sea intuitiva y sencilla.

2.1. La Arquitectura de Tres Niveles (Three-Tier Architecture)

Las aplicaciones web modernas organizan su software siguiendo una **three-tier architecture**. Esta separación permite que cada nivel se ejecute en su propia infraestructura y pueda desarrollarse o escalarse de forma independiente:

- **1. Nivel de presentación:** Define la interfaz de usuario y la comunicación con la aplicación. Se ejecuta en el navegador del cliente mediante HTML, CSS y JavaScript.
- **2. Nivel de aplicación (lógico o intermedio):** Es el corazón de la aplicación. Procesa los datos recopilados en la presentación aplicando la lógica de negocio antes de consultar el nivel de datos. Se desarrolla con lenguajes como Python, Java, PHP o Node.js.
- **3. Nivel de datos:** Almacena y administra la información tratada mediante gestores de bases de datos relacionales (MySQL, PostgreSQL, Oracle) o NoSQL (MongoDB, Cassandra).

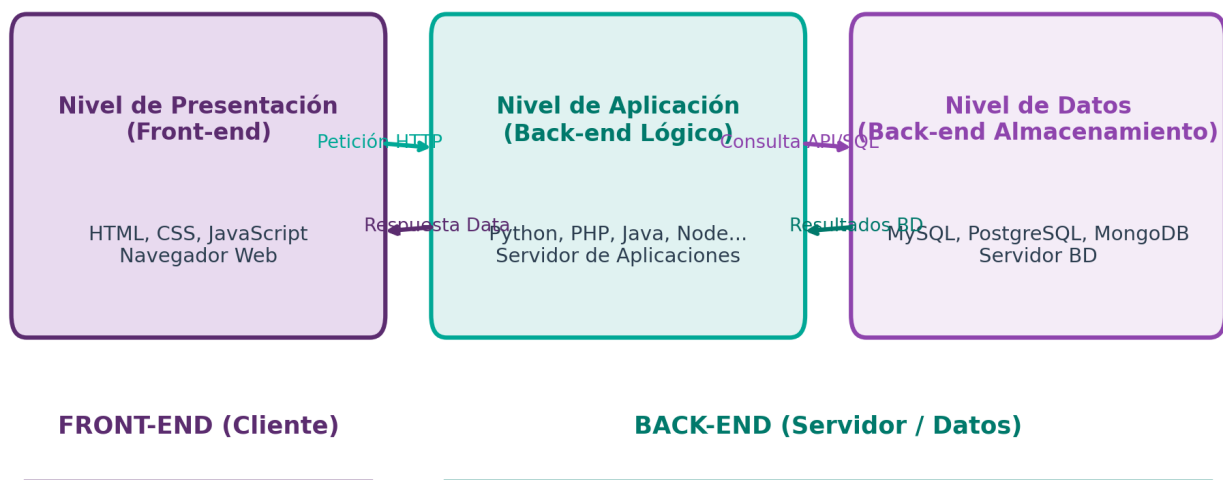


Figura 1.2 / 1.3. Esquema conceptual de la arquitectura de tres niveles y flujo de comunicación HTTP.

3. TECNOLOGÍAS Y ECOSISTEMA DEL LADO CLIENTE

El desarrollo web en el entorno cliente exige dominar imprescindiblemente tres tecnologías clave:

- **HTML (HyperText Markup Language):** Lenguaje de marcado que proporciona la estructura básica y el contenido del sitio web. Sus principales ventajas son la simplicidad y la compatibilidad universal.
- **CSS (Cascading Style Sheets):** Lenguaje utilizado para definir la presentación estética, colores, maquetación y diseño adaptable (responsive).
- **JavaScript:** El lenguaje de programación que aporta dinamismo, interactividad en tiempo real, manipulación del DOM y ejecución lógica local sin necesidad de recargar la página.

3.1. Preprocesadores y Frameworks JavaScript

Junto a las tres tecnologías base existen herramientas complementarias esenciales:

- **SASS:** Preprocesador de CSS que añade variables, funciones y anidamiento para facilitar la reutilización de código estilístico.
- **jQuery:** Librería JavaScript concisa que simplificó históricamente la manipulación del DOM y eventos.
- **Vue.js:** Framework progresivo y muy compacto para crear interfaces reactivas de manera rápida.
- **AngularJS / Angular:** Framework completo promovido por Google para aplicaciones estructuradas de gran tamaño.
- **React:** Librería/framework creado por Meta (Facebook), basado en componentes reutilizables y Virtual DOM, siendo el preferido actualmente en desarrollos de alto tráfico.

3.2. ¿Por qué JavaScript es el Lenguaje Dominante?

Alrededor del **95% al 99% de todas las aplicaciones web del mundo** utilizan JavaScript. Sus méritos principales son:

1. Fácil implementación e integración directa en documentos HTML.
2. Ejecución directa en cualquier navegador sin depender de servidores activos para responder.
3. Creación de interfaces dinámicas e interactivas que mejoran sustancialmente la experiencia de usuario (UX).
4. Carga asíncrona de contenidos (AJAX/Fetch) modificando el documento sin parpadeos ni recargas completas.
5. Capacidad de adaptación continua e incorporación de estándares modernos de programación.

Alternativas a JavaScript: En escenarios concretos existen lenguajes como *TypeScript* (superconjunto tipado de Microsoft), *Dart* (Google), *ELM* o *CoffeeScript*, los cuales se compilan o transpilan finalmente a JavaScript para su ejecución.

4. HERRAMIENTAS PARA EL DESARROLLO CLIENTE

4.1. El Navegador Web

El navegador es el entorno de ejecución fundamental del código cliente. Las estadísticas globales reflejan la adopción del mercado:

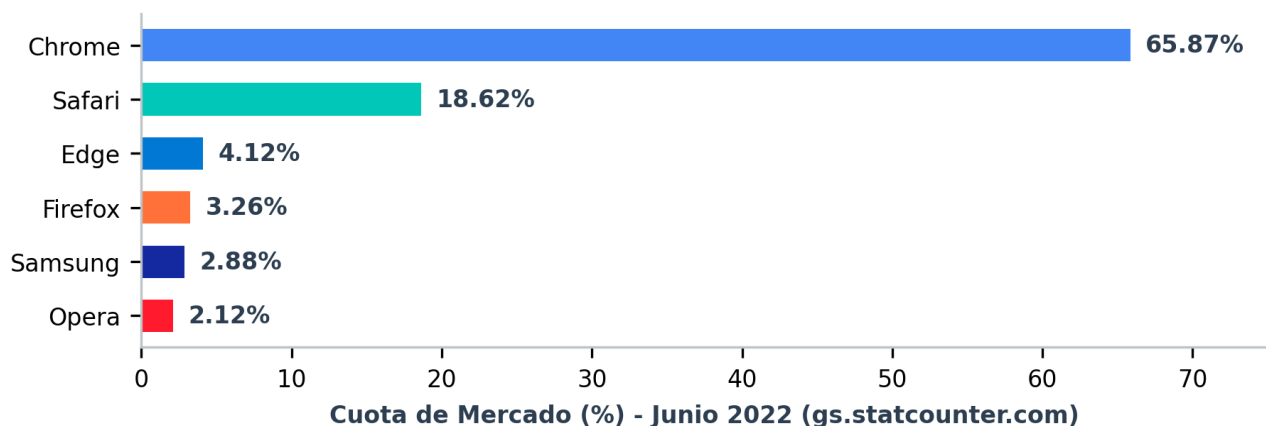


Figura 1.5. Cuota de mercado global de navegadores web (Fuente: StatCounter).

Implicación para desarrolladores: Aunque Chrome lidera con más del 65%, no se debe programar exclusivamente para él. El 35% restante representa millones de usuarios, por lo que es imprescindible probar las aplicaciones en los cuatro navegadores principales (Chrome, Safari, Edge, Firefox) y utilizar simuladores de pantallas multidispositivo.

4.2. Editores de Código vs. Entornos de Desarrollo Integrado (IDE)

Mientras que un **editor de código** es una herramienta ligera orientada a escribir e inspeccionar archivos rápidamente, un **IDE (Integrated Development Environment)** integra compiladores, depuradores avanzados, automatizadores de construcción y herramientas de control de versiones.

Características imprescindibles de un buen editor: Resaltado de sintaxis, autocompletado inteligente, detección de errores en tiempo real, navegación rápida de archivos, extensibilidad mediante complementos y soporte nativo para Git.

Herramientas populares en la industria:

- *Editores de código:* Visual Studio Code (el más utilizado por la comunidad), Sublime Text, Atom, Brackets.
- *IDEs completos:* WebStorm, Eclipse, NetBeans, Visual Studio.
- *Entornos online:* CodePen, JSFiddle, JSBin.

4.3. Intérprete de JavaScript

Todos los navegadores modernos incluyen un motor intérprete de JavaScript integrado. Para asegurar su correcto funcionamiento, debe verificarse que esté activado en las opciones de configuración de cada navegador (*Seguridad y Privacidad* -> *JavaScript* -> *Permitir*).

4.4. Ecosistema Avanzado: Node.js y Git

- **Node.js:** Plataforma basada en el motor V8 de Chrome que permite ejecutar JavaScript fuera del navegador (en el servidor). Ha revolucionado el desarrollo permitiendo crear arquitecturas *full-stack* totalmente unificadas en

un solo lenguaje.

- **Git:** El sistema de control de versiones distribuido estándar en la industria. Permite registrar el historial de cambios, trabajar en equipo mediante ramas (*branches*) y fusionar modificaciones sin perder código.

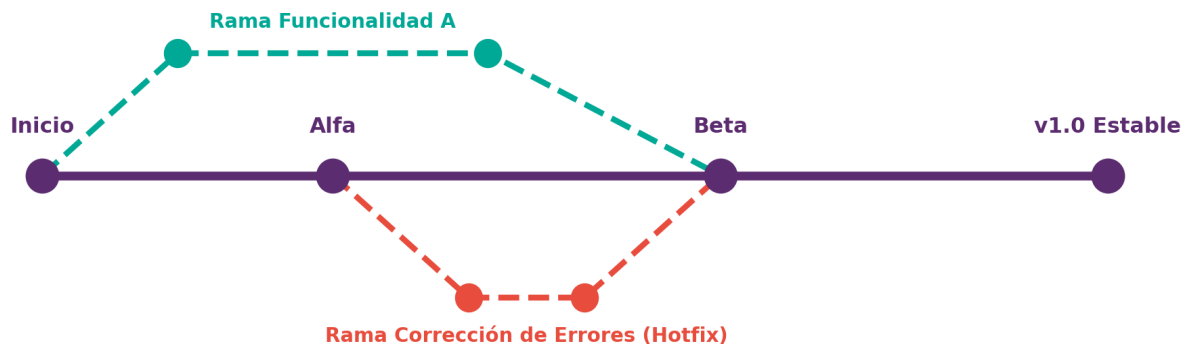


Figura 1.8. Diagrama gráfico de flujo de ramificaciones y fusiones de código en Git.

5. RESUMEN DEL TEMA Y MAPA CONCEPTUAL

Mapa Conceptual Sintetizado

JERARQUÍA GLOBAL DEL TEMA 1:

1. **Desarrollo de Software:** Ciclos de vida (Cascada, Iterativo, Incremental, Espiral, Prototipos) y Tipos de aplicaciones (Personalizadas, RAD, Low-code, Web).
2. **Lados de una Aplicación Web:** Front-end (Cliente) y Back-end (Servidor).
3. **Three-Tier Architecture:** Nivel de Presentación → Nivel de Aplicación → Nivel de Datos.
4. **Tecnologías Cliente:** HTML (Estructura), CSS/SASS (Estilos), JavaScript (Dinamismo), Frameworks (React, Vue, Angular).
5. **Herramientas de Trabajo:** Navegador Web (Chrome, Safari, Edge, Firefox), Editor/IDE (VS Code, Sublime), Intérprete JS, Node.js y Git.

6. ACTIVIDADES FINALES Y AUTOEVALUACIÓN

Autoevaluación - Test de Comprobación

1.1. ¿Cuál es el objetivo último del desarrollo de software?

- a) Crear empleo | b) Resolver una necesidad | c) Reducir costes | d) Promover la innovación.

1.2. ¿Cuál de las siguientes no es una fase del proceso de desarrollo de software?

- a) Definición de requisitos | b) Envío del presupuesto | c) Diseño de la arquitectura | d) Despliegue.

1.3. ¿En qué nivel de la three-tier architecture encaja el desarrollo del lado cliente?

- a) Nivel de datos | b) Nivel de presentación | c) Nivel de aplicación | d) Ninguno.

1.4. ¿Cuáles son las tres tecnologías imprescindibles para el desarrollo cliente?

- a) SASS, HTML, CSS | b) JavaScript, React, Vue | c) HTML, CSS, JavaScript | d) jQuery, Git, PHP.

1.5. ¿Qué característica no pertenece a JavaScript?

- a) Necesita conexión a internet para ejecutarse | b) Es fácil de implementar | c) Aporta dinamismo | d) Se ejecuta en cliente.

Actividades de Aplicación

1.13. Selecciona 4 tecnologías cliente razonadas para: a) Landing page de captación de correo; b) Web de noticias con votaciones en tiempo real; c) Sitio estático informativo.

1.14. Elabora una lista con las ventajas operativas y de rendimiento que supone la ejecución de código interactivo en el cliente sin recargar la página completa.